

But recent versions of Terraform have introduced a feature to import existing resources not created through Terraform into its *state* file. Currently, the import feature only updates the *state* file, but the user needs to create the configuration for the imported resources. Otherwise, Terraform will show the imported resources with no configuration and mark those for destruction.

Let's make this clear by importing an AWS instance, which wasn't brought up through Terraform, into some Terraform-created infrastructure. You need to run the command `terraform import aws_instances.<Terraform Resource Name> <id of the instance>` in the directory where a Terraform state file is located. After the successful import, Terraform gathers information about the instance and adds a corresponding section in the *state* file. If you see the Terraform plan now, it'll show something like what follows:

```
- aws_instance.<Terraform Resource Name>
```

So it means that now you need to create a corresponding configuration in an existing or new *.tf* file. In our example, the following Terraform section should be enough to not let Terraform destroy the imported resource.

```
resource "aws_instance" "<Terraform Resource Name>" {
  ami = "<AMI>"
  instance_type = "<Sizing info>"

  tags {
    ...
  }
}
```

Please note that you only need to mention the Terraform resource attributes that are required as per the Terraform document. Now, if you see the Terraform plan, the earlier shown destruction plan goes away for the imported resource. You could use the following command to extract the attributes of the imported resource to create its configuration:

```
sed -n '/aws_instance.<Terraform Resource Name>/,/}/p'
terraform.tfstate | \
grep -E 'ami|instance_type|tags' | grep -v '%' | sed 's/
"/' | sed 's/:/ =/'
```

Please pay attention when you import a resource into your current Terraform state and decide not to use that going forward. In which case, don't forget to rename your *terraform.state.backup* as *terraform.state* file to roll back to the previous state. You could also delete that resource block from your *state* file, as an alternative, but it's not a recommended approach. Otherwise, Terraform will try to delete the imported but not desired resource and that could be catastrophic in some cases.

The official Terraform documentation provides clear examples to import the various resources into an existing Terraform infrastructure. But if you are looking to include the existing AWS resources in the AWS infra created by Terraform in a more automated way, then take a look at the Terraforming tool link in the *References* section.

 **Note:** Terraform providers are no longer distributed as part of the main Terraform distribution. Instead, they are installed automatically as part of running terraform init. The *import* command requires that imported resources be specified in the configuration file. Please see [terraform changelog https://github.com/hashicorp/terraform/blob/v0.10.0/CHANGELOG.md](https://github.com/hashicorp/terraform/blob/v0.10.0/CHANGELOG.md) for these.

Missing bytes

You should now be feeling comfortable about starting to automate the provisioning of your cloud infrastructure. To be frank, Terraform is so feature-rich now that it can't be fully covered in a single or multiple articles and deserves a dedicated book (which has already shaped up in the form of an ebook, 'Terraform Up & Running'). So you could further take a look at the examples provided in its official Git repo. Also, the *References* section offers a few pointers to some excellent reads to make you more comfortable and confident with this excellent cloud provisioning tool.

Creating on-demand and scalable infrastructure in the cloud is not very difficult if some very simple basic principles are adopted and implemented using some feature-rich but no-fuss, easy-to-use standalone tools. Terraform is an indispensable tool for creating and managing cloud infrastructure in an idempotent way across a number of cloud providers. It could further be glued together with some other management pieces to create an immutable infrastructure workflow that can tame any kind of modern cloud infrastructure. The 'Terraform Up and Running' ebook is already out in the form of a print book. 

References

- [1] Terraform examples: <https://github.com/hashicorp/terraform/tree/master/examples>
- [2] Terraforming tool: <https://github.com/dtan4/terraforming>
- [3] A Comprehensive Guide to Terraform: <https://blog.gruntwork.io/a-comprehensive-guide-to-terraform-b3d32832baca#.ldays7wk>
- [4] Terraform Up & Running: <http://www.terraformupandrunning.com/?ref=gruntwork-blog-comprehensive-terraform>

By: Ankur Kumar

The author is a systems and infrastructure developer/architect and FOSS researcher, currently based in the US. You can find some of his other writings on FOSS at: <https://github.com/richnuseeeks>.